

React.js Development Course Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

This ReactJS course is designed to take learners from the fundamentals of modern frontend development to building production-ready, scalable React applications. The syllabus covers React fundamentals, component-driven development, Hooks, routing, forms, API integration, state management, performance optimization, testing, security, deployment, and modern development practices.

The course is suitable for students, freshers, frontend developers, full-stack developers, and working professionals who want to build modern web applications using ReactJS.

1. Prerequisites

Before starting ReactJS, learners should have a working understanding of:

HTML

- HTML document structure
- Semantic HTML
- Forms and input elements
- Tables
- Lists
- Links and navigation
- Images and multimedia
- HTML5 elements
- Accessibility basics

CSS

- CSS selectors
- Box model
- Display properties
- Positioning
- Flexbox
- CSS Grid
- Responsive design
- Media queries
- Pseudo-classes and pseudo-elements
- CSS variables
- Transitions and animations

JavaScript Fundamentals

- Variables: var, let, const
- Data types
- Operators
- Conditional statements
- Loops
- Functions
- Arrays
- Objects
- String methods
- Array methods
- Object methods

- Scope
- Hoisting
- Closures
- DOM basics
- Events
- Error handling

2. Modern JavaScript for React

React development heavily depends on modern JavaScript concepts.

ES6+ Features

- let and const
- Arrow functions
- Template literals
- Default parameters
- Rest parameters
- Spread operator
- Object shorthand
- Destructuring
- Optional chaining
- Nullish coalescing
- Modules
- Import and export
- Classes
- Promises
- Async/await

Array Methods

- map()
- filter()
- reduce()
- find()
- findIndex()
- some()
- every()
- forEach()
- sort()

Asynchronous JavaScript

- Callbacks
- Promises
- Promise chaining
- async
- await
- Fetch API
- Error handling with async operations

JavaScript Modules

- Named exports
- Default exports
- Import syntax
- Module organization

3. Introduction to ReactJS

React Fundamentals

- What is React?
- Why React?
- History and evolution of React
- Features of React
- Component-based architecture
- Declarative programming
- Virtual DOM
- Reconciliation
- Single Page Applications
- React ecosystem

React vs Traditional JavaScript

- DOM manipulation
- State-driven UI
- Component reuse
- Unidirectional data flow

React vs Other Frontend Frameworks

- React vs Angular
- React vs Vue
- React vs traditional JavaScript

4. React Development Environment

Required Tools

- Node.js
- npm
- npx
- VS Code
- Browser developer tools
- Git
- GitHub

Project Creation

- Understanding modern React project tooling
- Creating a React project with Vite
- Project structure
- Running the development server
- Building for production

Important Project Files

- `package.json`
- `src`
- `public`
- `index.html`
- Entry files
- Component files
- Configuration files

npm Fundamentals

- Installing packages

- Removing packages
- Dependencies
- Dev dependencies
- npm scripts
- Semantic versioning basics

5. JSX

Introduction to JSX

- What is JSX?
- JSX syntax
- JSX vs HTML
- JavaScript expressions inside JSX

JSX Rules

- Single parent element
- Closing tags
- className
- htmlFor
- CamelCase attributes

Expressions in JSX

- Variables
- Functions
- Mathematical expressions
- Conditional expressions
- Object properties

JSX Best Practices

- Readable JSX
- Fragment usage
- Avoiding deeply nested markup

6. React Components

Component Fundamentals

- What is a component?
- Functional components
- Component naming conventions
- Component reuse

Component Structure

- Creating components
- Exporting components
- Importing components
- Nested components

Component Composition

- Parent components
- Child components
- Reusable UI components
- Composition patterns

React Fragments

- Fragment syntax
- Short fragment syntax

7. Props

Props Fundamentals

- What are props?
- Passing data to components
- Reading props
- Multiple props

Destructuring Props

- Function parameter destructuring
- Default values

Different Prop Types

- Strings
- Numbers
- Booleans
- Arrays
- Objects
- Functions
- Components

Special Props

- children

Props Best Practices

- Immutability
- Clear component APIs
- Avoiding excessive prop drilling

8. Rendering Data

Rendering Variables

- Strings
- Numbers
- Expressions

Rendering Lists

- `map()`
- Dynamic lists
- Rendering objects

Keys

- Why keys are required
- Choosing stable keys
- Problems with array indexes

Conditional Rendering

- `if`
- Ternary operator
- Logical AND (`&&`)
- Early returns
- Conditional components

9. Event Handling

React Events

- onClick
- onChange
- onSubmit
- onFocus
- onBlur
- Keyboard events
- Mouse events

Event Handler Functions

- Inline handlers
- Named handlers
- Passing arguments

React Event Objects

- Event target
- Event values
- Preventing default behavior

Common Event Patterns

- Buttons
- Forms
- Inputs
- Interactive components

10. State in React

Understanding State

- What is state?
- Props vs state
- State-driven UI
- State immutability

useState

- Declaring state
- Reading state
- Updating state
- Initial state
- Multiple state variables

Updating State Correctly

- Previous-state callback
- Updating objects
- Updating arrays
- Adding items
- Removing items
- Updating list items

State Design

- Avoiding redundant state
- Derived state
- Choosing where state should live

11. Forms in React

Controlled Components

- Text inputs
- Number inputs
- Password inputs
- Textareas
- Select boxes
- Checkboxes
- Radio buttons

Form Handling

- Capturing values
- Form submission
- Preventing default submission
- Resetting forms

Multiple Form Fields

- Single change handler
- Dynamic field updates

Form Validation

- Required fields
- Length validation
- Email validation
- Password validation
- Custom validation

Form Libraries – Introduction

- React Hook Form
- Schema validation concepts
- Zod/Yup overview

12. React Hooks

Introduction to Hooks

- Why Hooks?
- Rules of Hooks
- Hook naming conventions

Core Hooks

- useState
- useEffect
- useContext
- useRef
- useReducer
- useMemo
- useCallback

Additional Hooks

- useId
- useLayoutEffect
- useImperativeHandle
- useTransition

- `useDeferredValue`

Modern React Hooks and APIs

- `use`
- `useOptimistic`
- `useActionState`
- Form-action-oriented patterns
- Concurrent UI concepts

13. `useEffect` and Side Effects

Side Effects

- What is a side effect?
- Effects vs event handlers

`useEffect`

- Running effects
- Dependency array
- Empty dependency array
- Effects based on state or props

Common Use Cases

- API calls
- Timers
- Event listeners
- DOM integration
- Synchronizing external systems

Cleanup Functions

- Removing listeners
- Clearing timers
- Preventing memory leaks

Common `useEffect` Mistakes

- Infinite loops
- Missing dependencies
- Unnecessary effects
- Derived state in effects

14. Component Lifecycle Concepts

Although functional components use Hooks, developers should understand lifecycle concepts.

Lifecycle Phases

- Mounting
- Updating
- Unmounting

Lifecycle with Hooks

- Mount behavior
- Dependency-based updates
- Cleanup on unmount

Strict Mode

- Purpose of Strict Mode

- Development behavior
- Detecting unsafe side effects

15. useRef

Reference Fundamentals

- Creating refs
- Accessing DOM elements

Common Use Cases

- Input focus
- Scroll control
- Video/audio controls
- Keeping mutable values

State vs Ref

- When to use state
- When to use refs

16. Context API

Context Fundamentals

- What is Context?
- Prop drilling problem
- Creating context
- Providing context
- Consuming context

useContext

- Accessing shared data

Common Use Cases

- Theme
- Authentication
- User preferences
- Language settings

Context Best Practices

- Avoiding unnecessary global state
- Splitting contexts
- Performance considerations

17. useReducer

Reducer Pattern

- State
- Actions
- Reducer function
- Dispatch

useReducer

- Initial state
- Action objects
- Updating complex state

useReducer vs useState

- When to use each

Combining Context and Reducer

- Lightweight application-wide state management

18. Custom Hooks

Custom Hook Fundamentals

- Why custom hooks?
- Naming conventions
- Extracting reusable logic

Examples

- useFetch
- useLocalStorage
- useDebounce
- useWindowSize
- useOnlineStatus

Best Practices

- Reusability
- Separation of concerns
- Avoiding hidden dependencies

19. Styling React Applications

Plain CSS

- Component CSS files
- Global CSS

Inline Styling

- React style objects
- Dynamic styles

CSS Modules

- Scoped styles
- Naming conventions

Utility-First CSS

- Tailwind CSS introduction
- Responsive utilities

CSS-in-JS Overview

- Styled-components concept
- Emotion concept

Styling Best Practices

- Component-level organization
- Responsive design
- Design consistency

20. React Router

Client-Side Routing

- SPA routing
- URL-based navigation

React Router Setup

- Router configuration
- Routes
- Route components

Navigation

- Link
- NavLink
- Programmatic navigation

Route Parameters

- Dynamic routes
- URL parameters

Query Parameters

- Reading search parameters

Nested Routes

- Parent and child routes
- Outlet concepts

Protected Routes

- Authentication guards
- Authorization basics

Error Pages

- 404 page
- Route-level error handling

21. API Integration

REST API Fundamentals

- HTTP methods
- GET
- POST
- PUT
- PATCH
- DELETE

HTTP Concepts

- Request
- Response
- Headers
- Status codes
- JSON

Fetch API

- Fetching data
- Posting data
- Updating data
- Deleting data

Axios

- Axios setup
- GET requests
- POST requests
- PUT/PATCH requests
- DELETE requests
- Axios instances
- Interceptors

API State Handling

- Loading state
- Success state
- Empty state
- Error state

Request Cancellation

- AbortController concepts

22. Data Fetching Libraries

Server State vs Client State

- Differences
- Caching requirements

TanStack Query / React Query

- Query setup
- useQuery
- useMutation
- Query keys
- Caching
- Refetching
- Loading/error states
- Query invalidation
- Pagination
- Optimistic updates

23. State Management

State Management Categories

- Local component state
- Lifted state
- Context state
- Global application state
- Server state

Redux Fundamentals

- Store
- State
- Actions
- Reducers
- Dispatch
- Selectors

Redux Toolkit

- Configuring store
- createSlice
- Reducers
- Actions
- useSelector
- useDispatch

Async Redux

- createAsyncThunk
- Async state
- Loading and error handling

Alternative State Management

- Zustand introduction
- Lightweight stores
- Store actions
- Selecting state

Choosing State Management Tools

- Context vs Redux Toolkit
- Redux vs Zustand
- Client state vs server-state libraries

24. Component Communication

Parent to Child

- Props

Child to Parent

- Callback props

Sibling Communication

- Lifting state up

Deep Component Communication

- Context API

Global Communication

- State management libraries

25. Reusable Component Design

Reusable UI Components

- Buttons
- Inputs
- Cards
- Modals
- Tables
- Alerts
- Dropdowns
- Loaders

Component API Design

- Props design
- Default values

- Variants
- Composition

Controlled vs Uncontrolled Components

Container and Presentational Concepts

Compound Components – Introduction

26. Advanced React Patterns

Composition

- Component composition
- children
- Slots-like patterns

Render Props – Concept

Higher-Order Components – Concept

Controlled Components

Uncontrolled Components

Compound Components

Headless Components

Provider Pattern

Custom Hooks Pattern

27. Error Handling

JavaScript Errors

- try
- catch
- finally

API Error Handling

- HTTP error states
- User-friendly messages

React Error Boundaries

- Purpose
- Error fallback UI
- Limitations

Route-Level Error Handling

28. Performance Optimization

React Rendering

- Render cycles
- State updates
- Parent-child re-rendering

Memoization

- `React.memo`
- `useMemo`
- `useCallback`

Code Splitting

- Dynamic imports
- `React.lazy`
- `Suspense`

List Performance

- Stable keys
- Pagination
- Infinite scrolling
- Virtualization concepts

Other Optimization Techniques

- Debouncing
- Throttling
- Avoiding unnecessary state
- Efficient component design
- Image optimization
- Bundle analysis

29. Suspense and Concurrent UI

Suspense

- Loading boundaries
- Fallback UI
- Lazy-loaded components

Concurrent Rendering Concepts

- Responsive updates
- Interruptible rendering concepts

Transitions

- `useTransition`
- Non-urgent updates

Deferred Values

- `useDeferredValue`

Optimistic UI

- Optimistic updates
- `useOptimistic`

30. Authentication and Authorization

Authentication Concepts

- Login
- Logout
- Registration
- Sessions
- Tokens

Token-Based Authentication

- Access tokens
- Refresh tokens
- JWT concepts

Authentication Flow in React

- Login forms
- Persisting authentication
- Auth context/store

Protected Routes

- Restricting unauthenticated users

Role-Based Authorization

- User roles
- Admin routes
- UI permission checks

Security Considerations

- Avoiding insecure token handling
- HTTPS
- XSS basics
- CSRF basics
- Secure API design awareness

31. Browser Storage

Local Storage

- Storing values
- Retrieving values
- Removing values

Session Storage

Cookies – Overview

Storage Use Cases

- Themes
- Preferences
- Temporary application data

Security Considerations

- Sensitive data
- Authentication storage risks

32. TypeScript with React

TypeScript Fundamentals for React

- Primitive types
- Arrays
- Objects
- Interfaces
- Type aliases
- Unions
- Generics

Typing Components

- Props types
- Optional props
- Children

Typing Hooks

- useState
- useRef
- useReducer

Typing Events

- Change events
- Click events
- Form events

API Types

- Request/response models

TypeScript Best Practices

- Avoiding excessive any
- Reusable interfaces and types

33. Testing React Applications

Testing Fundamentals

- Unit testing
- Integration testing
- End-to-end testing

Vitest / Jest Concepts

- Test suites
- Test cases
- Assertions
- Mock functions

React Testing Library

- Rendering components
- Finding elements
- User interactions
- Testing forms
- Testing conditional rendering

Mocking APIs

- Mock functions
- Mock Service Worker concepts

End-to-End Testing

- Playwright or Cypress overview
- Testing user flows

Testing Best Practices

- Test behavior, not implementation
- Accessible selectors
- Meaningful coverage

34. Debugging React Applications

Browser Developer Tools

- Console
- Network tab
- Elements tab
- Application tab

React Developer Tools

- Component tree
- Props inspection
- State inspection
- Profiler

Common React Problems

- Infinite renders
- Incorrect dependencies
- Undefined props
- State mutation
- Async race conditions
- Key warnings

35. Accessibility

Accessibility Fundamentals

- Semantic HTML
- Keyboard navigation
- Focus management
- Labels

ARIA Basics

- Roles
- Labels
- States

Accessible Forms

- Label association
- Validation messages

Accessible Components

- Modal dialogs
- Dropdowns
- Buttons
- Navigation

36. Security Best Practices

Frontend Security Fundamentals

- XSS awareness
- Avoiding dangerous HTML injection
- Input validation
- Output encoding concepts

API Security Awareness

- HTTPS
- Authentication

- Authorization

Environment Variables

- Public frontend configuration
- Why secrets should not be stored in frontend code

Dependency Security

- npm audit concepts
- Updating dependencies

37. React Project Architecture

Folder Structures

- Feature-based organization
- Component-based organization

Common Directories

- components
- pages
- features
- hooks
- services
- utils
- types
- assets
- context
- store

Separation of Concerns

- UI
- Business logic
- API layer
- State layer

Scalable Architecture

- Feature modules
- Shared components
- Shared utilities

38. API Service Layer

Creating API Services

- Centralized API client
- Base URL configuration
- Request methods

Axios Instances

- Default headers
- Timeouts
- Interceptors

Error Handling

- Centralized errors
- Unauthorized responses

Environment Configuration

- Development APIs
- Staging APIs
- Production APIs

39. Environment Management

Environment Variables

- .env
- Development environment
- Test environment
- Production environment

Vite Environment Variables

- Configuration
- Public environment variables

Environment-Based API URLs

Secure Configuration Principles

40. Git and GitHub for React Developers

Git Fundamentals

- Repository
- Clone
- Add
- Commit
- Push
- Pull
- Branch

Branching

- Feature branches
- Bug-fix branches

Collaboration

- Pull requests
- Code reviews
- Merge conflicts

GitHub Portfolio

- Repository organization
- README files
- Project screenshots
- Live demo links
- Meaningful commit history

41. Code Quality and Development Standards

Code Formatting

- Prettier

Linting

- ESLint

- Common rules

Naming Conventions

- Components
- Hooks
- Files
- Variables
- Functions

Clean Code

- Small components
- Reusable logic
- Separation of responsibilities
- Avoiding duplicate code

Code Review Practices

42. UI Libraries and Component Systems

Popular UI Libraries – Overview

- Material UI
- Ant Design
- Chakra UI
- React Bootstrap
- shadcn/ui concepts

Component Library Concepts

- Design systems
- Reusable components
- Theme configuration

Choosing a UI Library

- Project requirements
- Customization
- Accessibility
- Bundle size
- Team familiarity

43. Responsive React Applications

Responsive Layouts

- Flexbox
- Grid
- Media queries

Mobile-First Development

Responsive Components

Responsive Navigation

Responsive Tables and Cards

Cross-Browser Testing

44. Progressive Web App Concepts

PWA Fundamentals

- Progressive enhancement
- Installable web apps
- Offline concepts

Service Workers – Introduction**Web App Manifest****Caching Strategies – Overview****45. React and Backend Integration****Connecting React with Backend Technologies**

- Node.js/Express
- Java/Spring Boot
- Python/FastAPI/Django
- .NET Web API

Frontend-Backend Workflow

- API contracts
- JSON
- CORS
- Authentication
- Error responses

CORS

- What is CORS?
- Common CORS issues
- Development vs production configuration

46. Deployment**Production Build**

- Build command
- Production assets
- Environment variables

Deployment Platforms

- Vercel
- Netlify
- Azure Static Web Apps
- AWS hosting concepts
- Firebase Hosting
- Traditional web servers

CI/CD Concepts

- Automated builds
- Automated testing
- Deployment pipelines

Environments

- Development
- Staging
- Production

47. React Development with AI Assistance

AI-Assisted Development

- Using coding assistants effectively
- Generating component scaffolding
- Explaining unfamiliar code
- Refactoring components
- Generating test cases
- Debugging errors
- Creating documentation

Prompting for React Development

- Giving project context
- Specifying component contracts
- Requesting typed code
- Asking for tests
- Asking for accessibility improvements

AI Code Review

- Checking generated code
- Security review
- Performance review
- Validating dependencies
- Avoiding blind copy-paste

48. Mini Projects

Students should complete multiple hands-on mini projects.

Mini Project 1 – Counter Application

Concepts:

- Components
- Events
- useState

Mini Project 2 – Todo Application

Concepts:

- State
- Lists
- Forms
- CRUD operations

Mini Project 3 – Weather Application

Concepts:

- API integration
- Loading states
- Error handling

Mini Project 4 – Expense Tracker

Concepts:

- Forms
- State management
- Filtering

- Calculations

Mini Project 5 – Product Catalog

Concepts:

- API calls
- Search
- Filters
- Sorting
- Routing

Mini Project 6 – Authentication UI

Concepts:

- Login
- Registration
- Protected routes
- Form validation

Mini Project 7 – Admin Dashboard

Concepts:

- Reusable components
- Charts
- Tables
- Routing
- Responsive UI

49. Major Project / Capstone Project

Students should build at least one production-style application.

Suggested Capstone Projects

E-Commerce Application

Features:

- Product listing
- Product details
- Search
- Filtering
- Cart
- Authentication
- Checkout UI
- Orders
- Admin dashboard

Learning Management System

Features:

- Student login
- Course catalog
- Course details
- Enrollment
- Progress tracking
- Instructor dashboard
- Admin dashboard

Job Portal

Features:

- Job listings
- Search
- Filters
- Candidate profile
- Employer dashboard
- Application management

Project Management Application

Features:

- Authentication
- Projects
- Tasks
- Status boards
- Team members
- Comments
- Dashboard

Social Media Application

Features:

- Authentication
- User profiles
- Posts
- Likes
- Comments
- Follow system
- Notifications
- Feed

50. Capstone Project Requirements

The final project should demonstrate:

- Responsive UI
- Component-based architecture
- React Hooks
- React Router
- Forms and validation
- REST API integration
- Authentication
- Authorization
- Global state management
- Server-state management where appropriate
- Loading and error handling
- Reusable components
- Clean folder structure
- TypeScript
- Testing
- Git/GitHub usage
- Production deployment
- Documentation
- README

- Screenshots
- Live project link

51. Interview Preparation

React Interview Topics

- Virtual DOM
- Reconciliation
- JSX
- Components
- Props
- State
- Hooks
- useState
- useEffect
- useRef
- useContext
- useReducer
- useMemo
- useCallback
- Custom hooks
- Controlled components
- Uncontrolled components
- Routing
- Context API
- Redux
- React Query
- Performance optimization
- Code splitting
- Suspense
- Error boundaries
- Authentication
- Testing

JavaScript Interview Topics

- Scope
- Hoisting
- Closures
- Event loop
- Promises
- Async/await
- Array methods
- Objects
- this
- Prototypes
- ES modules

Practical Interview Exercises

- Build a reusable component
- Build a search/filter interface
- Fetch and display API data
- Implement pagination
- Create a form with validation

- Debug a React component
- Optimize unnecessary renders
- Explain component architecture

52. Coding Exercises

Students should practice:

- Counter
- Toggle component
- Accordion
- Tabs
- Modal
- Dropdown
- Search box
- Debounced search
- Pagination
- Infinite scroll
- Form validation
- Todo CRUD
- Shopping cart
- Theme switcher
- Stopwatch
- Countdown timer
- Dynamic table
- API list
- Filterable products
- Authentication flow
- Protected routes

53. Recommended Development Stack

A modern React learning stack may include:

- React
- JavaScript ES6+
- TypeScript
- Vite
- React Router
- React Hook Form
- Zod
- Axios
- TanStack Query
- Redux Toolkit or Zustand
- Tailwind CSS or a component library
- Vitest
- React Testing Library
- Playwright or Cypress
- Git
- GitHub
- ESLint
- Prettier

54. Suggested Course Sequence

Phase 1 – Web and JavaScript Foundation

- HTML refresher
- CSS refresher
- JavaScript fundamentals
- Modern JavaScript
- Async JavaScript

Phase 2 – React Fundamentals

- React introduction
- JSX
- Components
- Props
- Events
- State
- Lists
- Conditional rendering
- Forms

Phase 3 – React Hooks

- useState
- useEffect
- useRef
- useContext
- useReducer
- Custom hooks

Phase 4 – Application Development

- React Router
- REST APIs
- Axios
- Authentication
- Form validation
- Browser storage

Phase 5 – Advanced State and Data

- Redux Toolkit
- Zustand
- TanStack Query
- Optimistic updates

Phase 6 – Production React

- TypeScript
- Testing
- Performance optimization
- Accessibility
- Security
- Project architecture

Phase 7 – Professional Development

- Git and GitHub
- Code quality
- Deployment
- CI/CD
- AI-assisted development

Phase 8 – Career Preparation

- Mini projects
- Capstone project
- GitHub portfolio
- Interview preparation
- Mock interviews

55. Learning Outcomes

After completing this ReactJS course, learners should be able to:

- Build modern single-page applications using React
- Create reusable and maintainable components
- Work confidently with JSX, props, state, and events
- Use React Hooks effectively
- Build and validate forms
- Implement client-side routing
- Consume REST APIs
- Handle loading and error states
- Implement authentication and authorization
- Manage local, global, and server state
- Use Redux Toolkit, Zustand, or Context appropriately
- Work with TanStack Query
- Develop React applications using TypeScript
- Write unit, integration, and end-to-end tests
- Optimize application performance
- Build accessible and responsive interfaces
- Organize scalable React projects
- Use Git and GitHub professionally
- Deploy React applications
- Build portfolio-ready projects
- Prepare for React frontend developer interviews

56. Suggested Course Deliverables

By the end of the course, each learner should ideally have:

- Multiple coding assignments
- Reusable React components
- 5–7 mini projects
- At least one major capstone project
- GitHub repositories
- Professional project README files
- Deployed applications
- React interview question preparation
- JavaScript interview preparation
- Mock interview experience
- AI-assisted development practice
- A frontend development portfolio

57. Optional Advanced Modules

For learners who want to continue beyond core React:

Next.js

- File-based routing
- Server rendering concepts
- Static generation concepts
- Server Components
- API/backend integration
- Full-stack React development

Advanced State Management

- Complex Redux architecture
- State normalization
- Middleware
- Persistence

Micro Frontends

- Architecture concepts
- Independent deployments
- Module federation concepts

Design Systems

- Reusable component libraries
- Storybook
- Tokens
- Documentation

GraphQL

- GraphQL concepts
- Queries
- Mutations
- Apollo Client overview

Real-Time Applications

- WebSockets
- Socket.IO
- Live notifications
- Chat applications

Advanced Performance

- Profiling
- Bundle optimization
- Rendering optimization
- Virtualized lists

58. Career Roles After ReactJS Training

This course can help learners prepare for roles such as:

- ReactJS Developer
- Frontend Developer
- UI Developer
- JavaScript Developer
- Web Application Developer
- Frontend Software Engineer
- Full-Stack Developer
- React TypeScript Developer
- Next.js Developer

Conclusion

ReactJS is one of the core technologies used for building modern frontend applications. A complete React developer should know more than just components and Hooks—they should understand modern JavaScript, API integration, routing, state management, TypeScript, testing, performance, accessibility, security, deployment, Git, and professional project development.

This syllabus is designed to provide a structured path from beginner-level React development to production-ready application development and interview preparation.

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.